

Architectural Design

Architectural design

- **What it is:** Architectural design is about planning how a software system should be put together and deciding its overall structure.
- **Why it matters:** It **connects requirements** (what the system should do) **with design** (how it should be built). It focuses on the main parts of the system and how they interact.
- **End result:** The process **produces a model (a kind of blueprint)** that shows how the system is made up of different parts that work and communicate with each other.

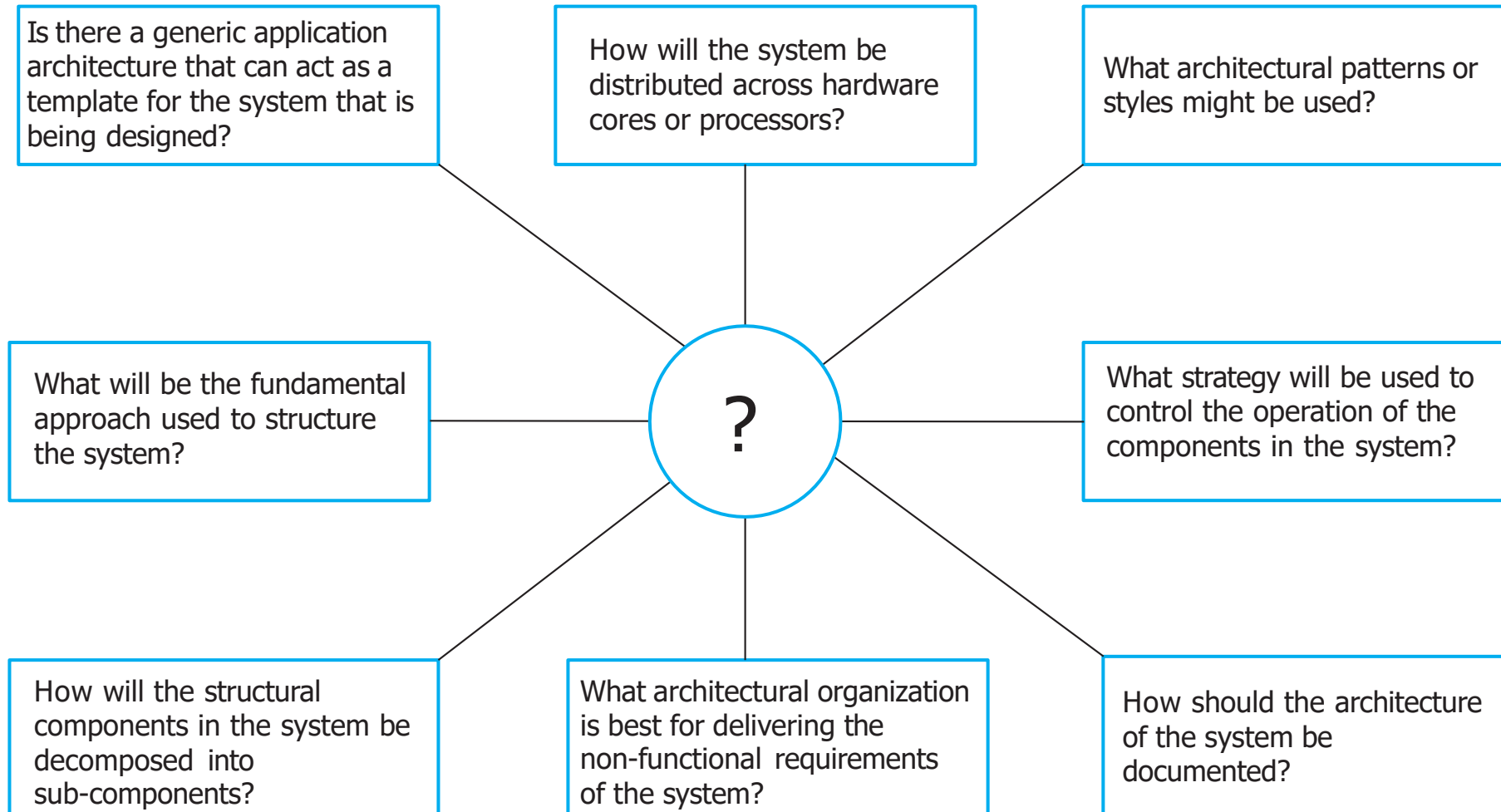
Architectural representations

- ⚙️ Software architectures are most often documented using informal block diagrams, where entities (components, subsystems) and their relationships are illustrated.
 - 👉 Very abstract - they do not show the nature of component relationships nor the externally visible properties of the sub-systems.
 - 👉 A major criticism is that such diagrams lack formal semantics. They do not specify the types of interactions between entities nor provide clarity about the functional or non-functional properties of architectural elements.
 - 👉 Despite these limitations, block diagrams remain valuable communication tools. They are easy for stakeholders to understand and serve as a foundation for early project planning and discussion before moving into more formal models.

Use of architectural models

- ⚙️ As a way of facilitating discussion about the system design
 - 👉 A high-level architectural view of a system is useful for communication with system stakeholders and project planning because it is not cluttered with detail. Stakeholders can relate to it and understand an abstract view of the system. They can then discuss the system as a whole without being confused by detail.
- ⚙️ As a way of documenting an architecture that has been designed
 - 👉 The aim here is to produce a complete system model that shows the different components in a system, their interfaces and their connections.

Architectural design decisions



ACU is building a student-placement portal used by students, university staff and industry supervisors

each pair to choose the two questions on the slide that they would investigate first and explain why.

Architecture design decisions

1. Use of Generic Architectures

Question: Is there a **reference model** or **generic architecture** that can be reused (e.g., client-server, layered architecture)?

Why: Reusing proven templates saves effort, reduces risk, and ensures best practices.

2. Distribution Across Hardware

Question: How will the system be **deployed across processors, servers, or cores**?

Why: Distribution decisions affect performance, scalability, and fault tolerance (e.g., cloud vs. on-premise, centralised vs. distributed).

3. Architectural Styles or Patterns

Question: Which **architectural style** (e.g., layered, microservices, event-driven, service-oriented) should be applied?

Why: Patterns influence modularity, reusability, scalability, and maintainability

4. System Structuring Approach

Question: What will be the **fundamental structuring principle** (e.g., modular decomposition, object-oriented, data-flow based)?

Why: The structural choice dictates how the system's complexity is managed.

Architecture design decisions

5. Component Decomposition

Question: How should large system components be **broken down into sub-components**?

Why: Decomposition improves clarity, testability, and team-level development.

6. Control Strategy

Question: How will **control and coordination** between components be managed (e.g., centralised control, peer-to-peer, event-driven)?

Why: This ensures predictable system behaviour and efficient execution.

7. Non-Functional Requirements

Question: What architectural choices best address **non-functional requirements** such as performance, security, reliability, and scalability?

Why: Non-functional needs often shape the architecture more than functional ones.

8. Documentation

Question: How will the architecture be **described and communicated** (e.g., block diagrams, UML, formal ADLs)?

Why: Good documentation supports stakeholder communication, system maintenance, and long-term evolution.

Architecture reuse

- ⚙ Systems in the same domain often have similar architectures that reflect domain concepts.
- ⚙ Application product lines are built around a core architecture with variants that satisfy particular customer requirements.
- ⚙ The architecture of a system may be designed around one of more architectural patterns or 'styles'.
 - ☞ These capture the essence of an architecture and can be instantiated in different ways.

Architecture and system characteristics

⚙️ Performance

- 👉 Localise critical operations and minimise communications. Use large rather than fine-grain components.

⚙️ Security

- 👉 Use a layered architecture with critical assets in the inner layers.

⚙️ Safety

- 👉 Localise safety-critical features in a small number of sub-systems.

⚙️ Availability

- 👉 Include redundant components and mechanisms for fault tolerance.

⚙️ Maintainability

- 👉 Use fine-grained, replaceable components.

“A medical-alert system must respond within 500 milliseconds and remain available during one node failure. Which two qualities dominate, and what architectural tactics might follow?”

“under what workload, failure and change scenario?”

“Because different stakeholders care about different qualities, one architecture diagram is rarely enough.”

Architectural patterns

An architectural pattern in software engineering is a **reusable solution to a recurring problem in software architecture**. It provides a **high-level template** that guides how systems should be structured and how their components should interact.

- It is **not code**, but a **design blueprint** that shapes the overall system structure.
- Patterns capture **proven practices**, tested in multiple projects and environments.
- Each pattern typically describes:
- **Problem** – the recurring design challenge.
- **Solution** – the structure and interactions that address the problem.
- **Context** – when the pattern is applicable.
- **Consequences** – benefits and trade-offs of using it.

Typical architectural patterns

- ⚙️ **Model View Controller:** separates domain state, presentation and input handling.
- ⚙️ **Layered architecture:** Layered architecture organises responsibilities and permitted dependencies by level.
- ⚙️ **Repository architecture:** Repository architecture coordinates subsystems through a shared data store.
- ⚙️ **Client-server architecture:** Client–server architecture separates service providers from service requesters.

identify the likely pattern from one phrase:

- “many tools share one project store”;
- “browser requests a service”;
- “UI separated from domain state”;
- “presentation calls business logic, which calls data access”.

The Model-View-Controller (MVC) pattern

Origins: It was introduced by *Smalltalk developers* but is now a **core pattern in modern frameworks** (e.g., Django, Ruby on Rails, ASP.NET, Spring MVC).

Concept:

- **Model:** Manages *data and business logic*. - it is more than “the database”.
- **View:** Handles *presentation of data* to the user.
- **Controller:** Manages *user input and interaction*, updating the model and view.

•Benefits:

- Promotes **separation of concerns** → easier to test, maintain, and scale.
- Multiple views can be supported from the same model (e.g., web view, mobile view).

•Challenges:

- Can add complexity for small applications.
- Requires careful coordination between the three parts.

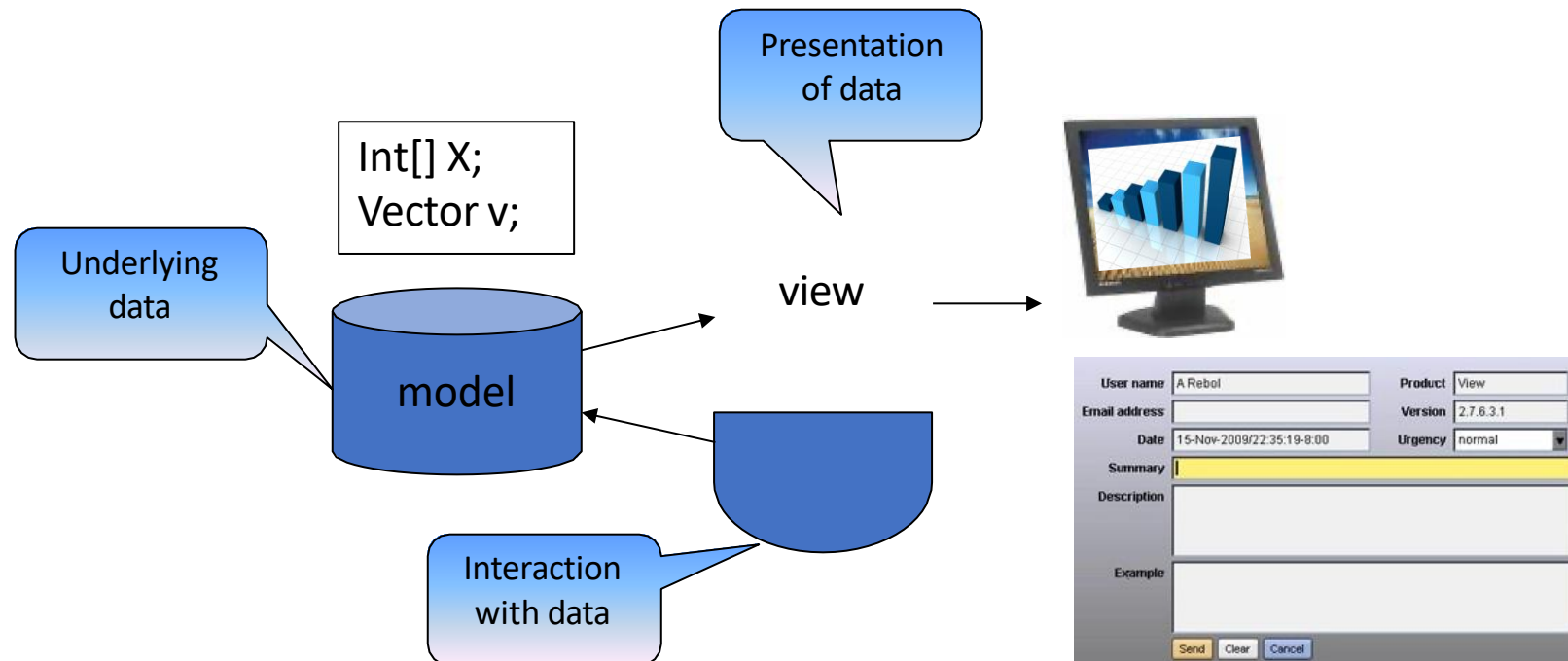
In an online banking transfer.

“Where should the rule ‘the transfer cannot exceed the available balance’ live?”

The Model-View-Controller (MVC) pattern

Real-world Example:

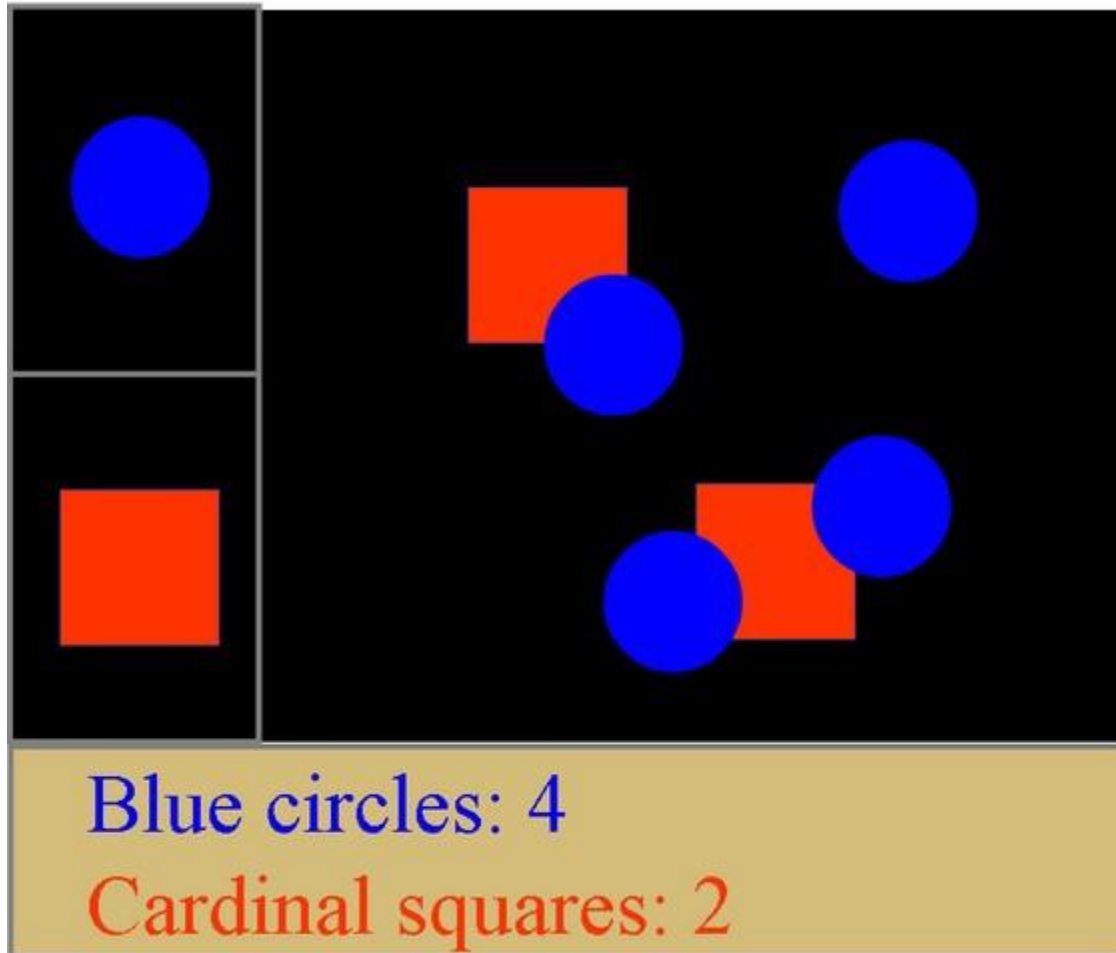
- Online shopping system:
 - *Model* → product inventory, orders, customer records.
 - *View* → product pages, shopping cart display.
 - *Controller* → handles “add to cart”, “checkout”, “payment processing” actions.



Question:
where three items
belong:

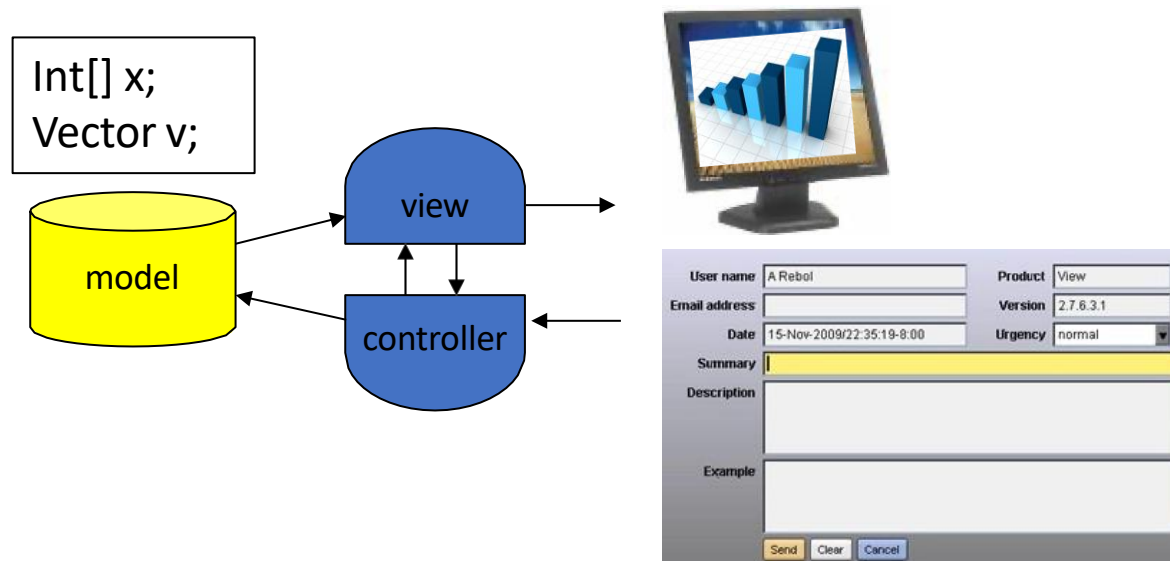
- product price calculation,
- button colour,
- and validation that ordered quantity is available

Example application



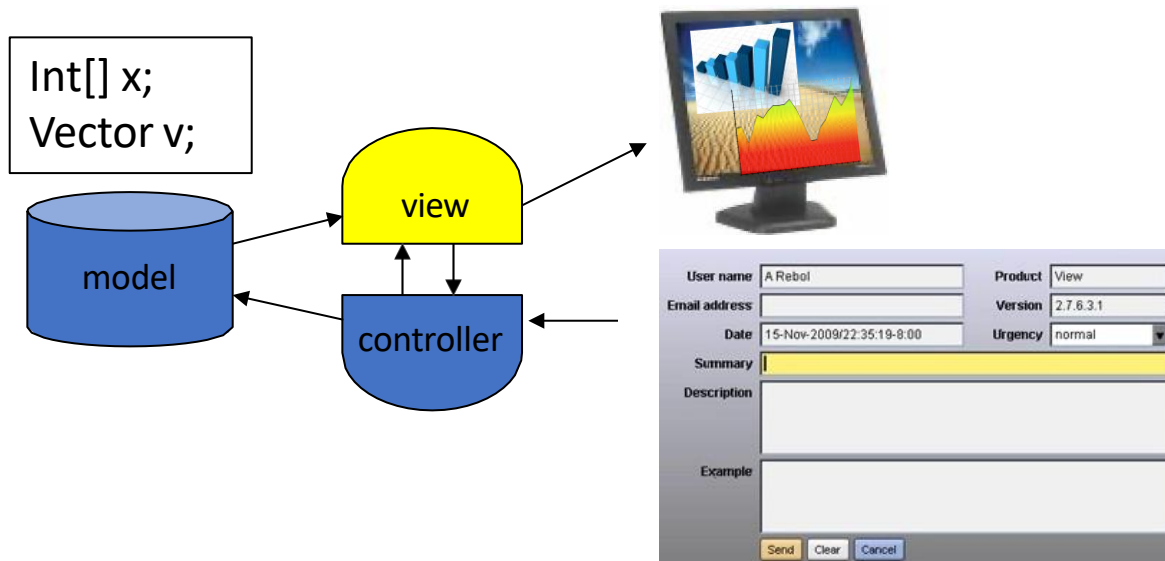
Model

- ⚙ Information the application is trying to manipulate
- ⚙ Representation of real-world objects using data structures
 - ☞ Shapes in a drawing program
 - ☞ List of people in a contact management program



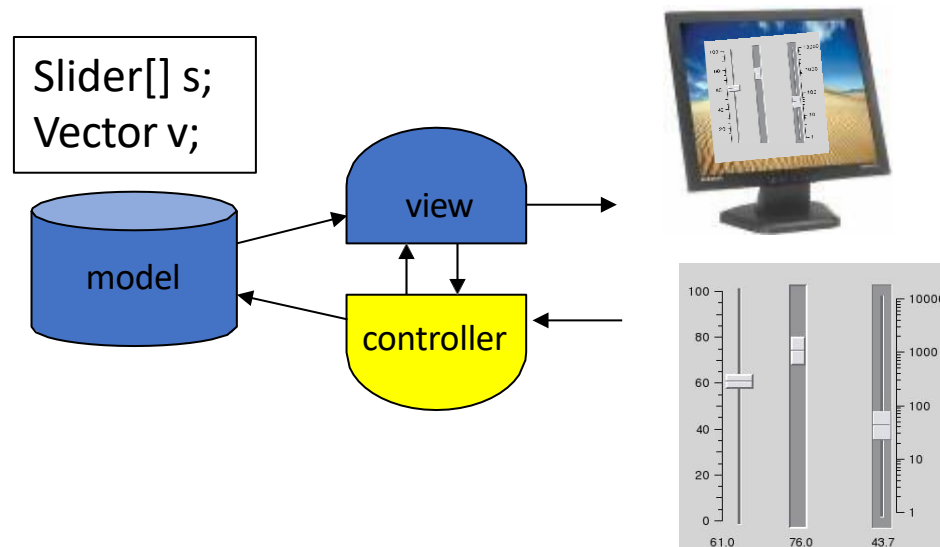
View

- ⚙ Implements a visual display of the model
- ⚙ May have multiple views
- ⚙ Any time the model is changed, each view must be notified so that it will be updated

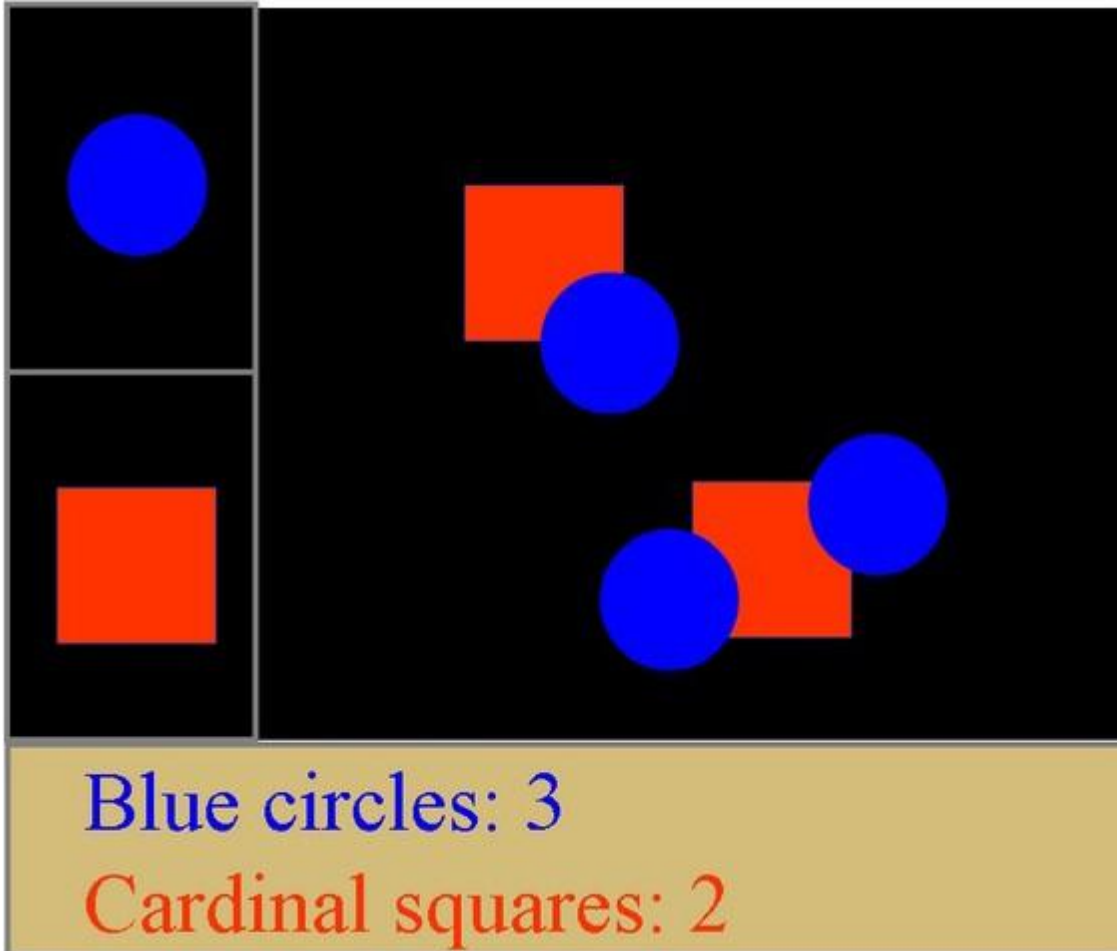


Controller

- ⚙ Receives all input events from the user
- ⚙ Decides what they mean and what to do
 - 👉 Communicates with view to determine the objects being manipulated (e.g., selection)
 - 👉 Calls model methods to make changes to objects
 - 👉 Updates views accordingly



Controller



“What is the minimum information an input event must supply to add another blue circle?”

what the Controller needs to know when the user clicks.

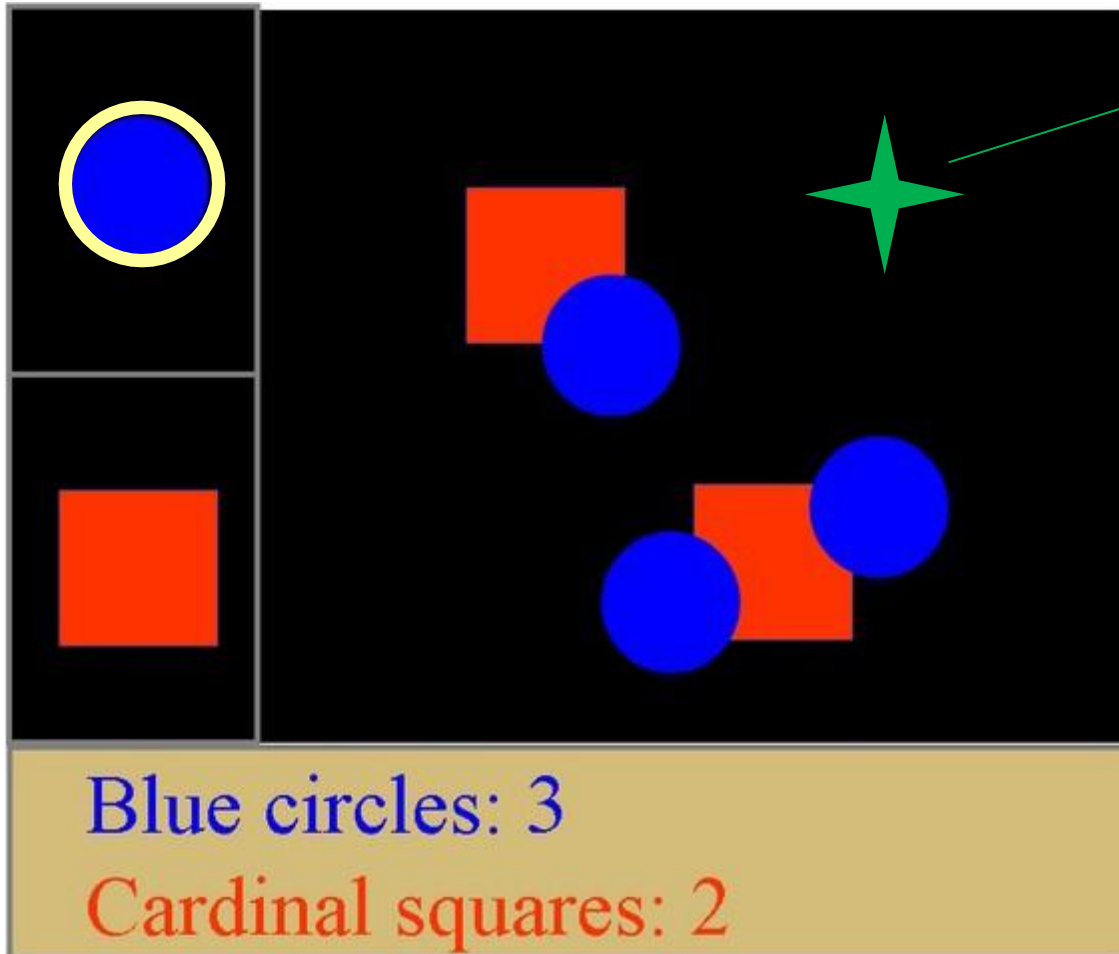
What is being created? → a circle

Where should it be created? → the mouse click coordinates, for example (x = 420, y = 180)

What colour/style? → blue

Controller

Here the blue-circle tool is selected and the user clicks.



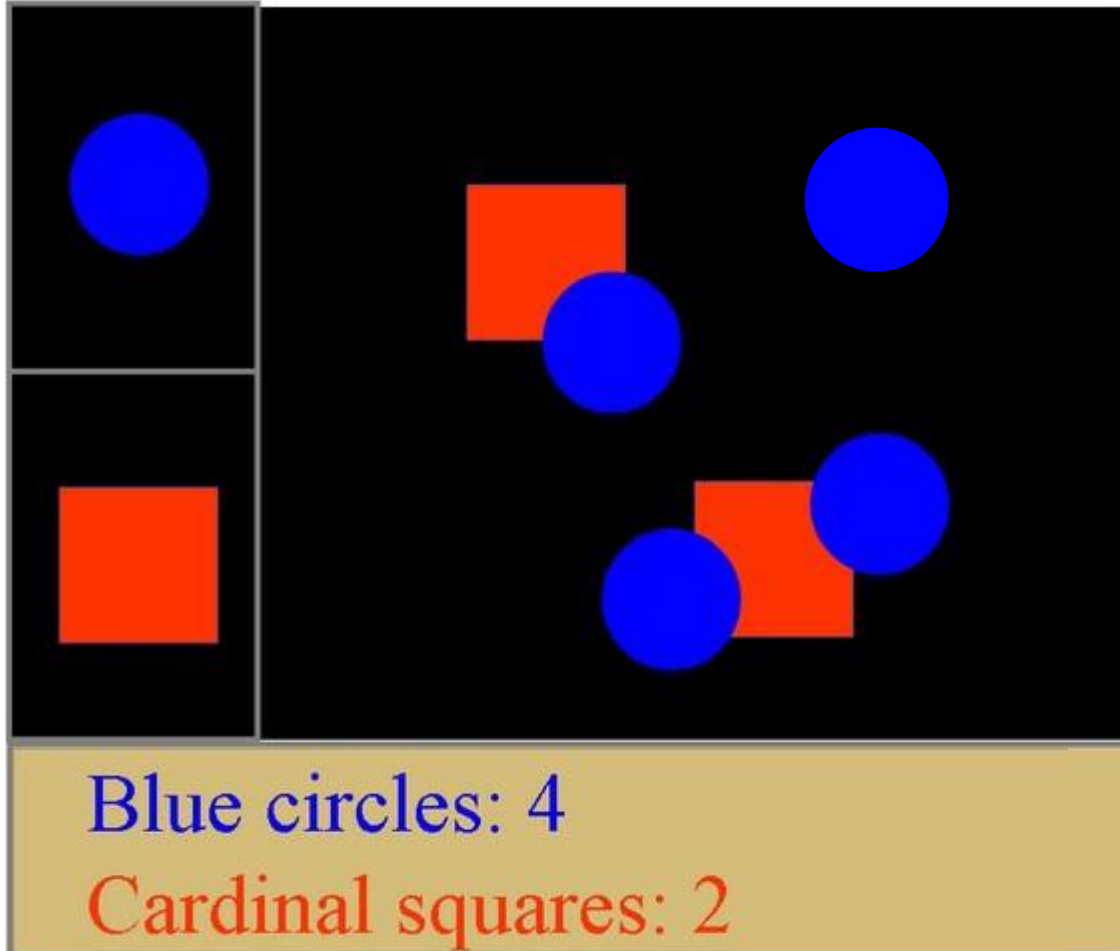
Click!

“What should happen if the click is outside the permitted drawing area?”

The Controller can perform the first check because it receives the user's input. But important rules should also be enforced by the Model. Otherwise another part of the application could bypass the Controller and create invalid data

Controller

Now the new blue circle has appeared.



name the four steps

Combining View & Controller

⚙️ View and controller are tightly integrated

- 👉 Lots of communication between the two, e.g. determine what was clicked on
- 👉 Almost always occur in pairs, i.e., for each view, need a separate controller

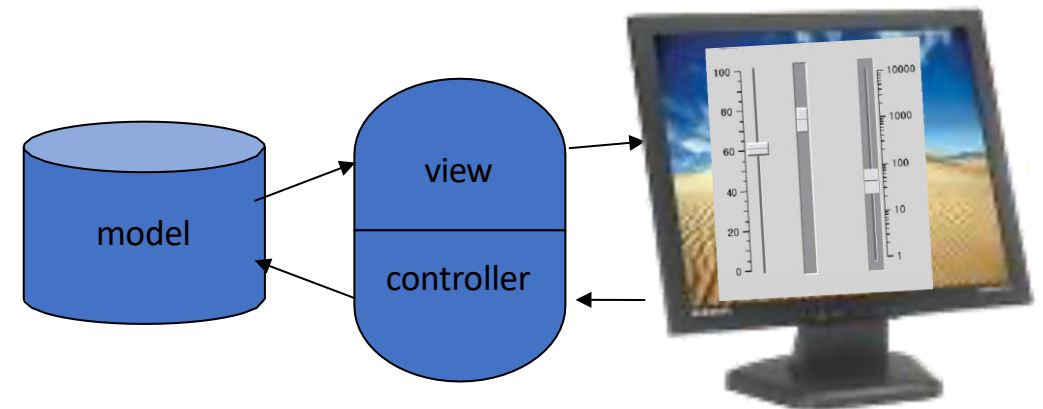
⚙️ Many architectures combine into a single unit

👉 Model

- 👉 maintain data, list of views
- 👉 notify views of changes to data

👉 View

- 👉 query model for data to present
- 👉 operate on model in response to input

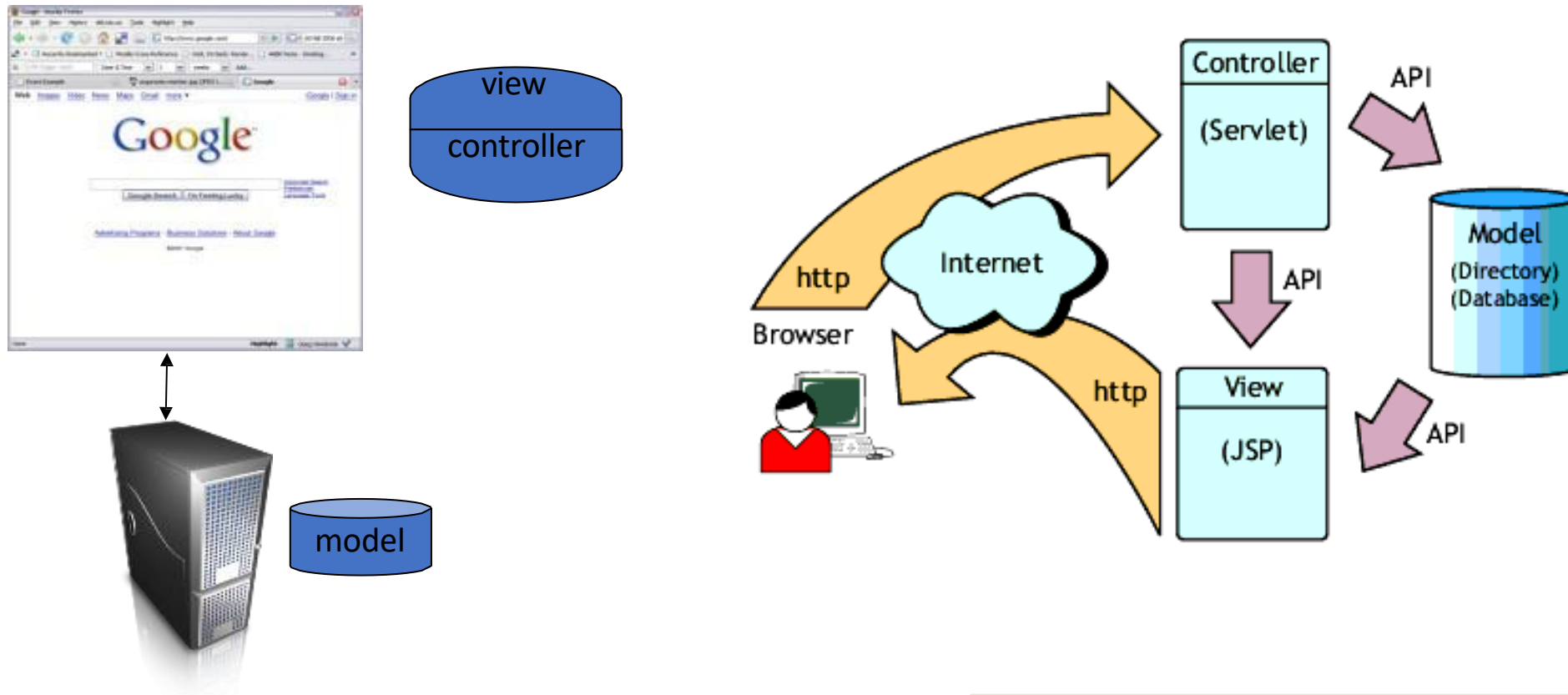


The Model-View-Controller (MVC) pattern

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model.
Example	The architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code and code complexity when the data model and interactions are simple.

MVC and the Web

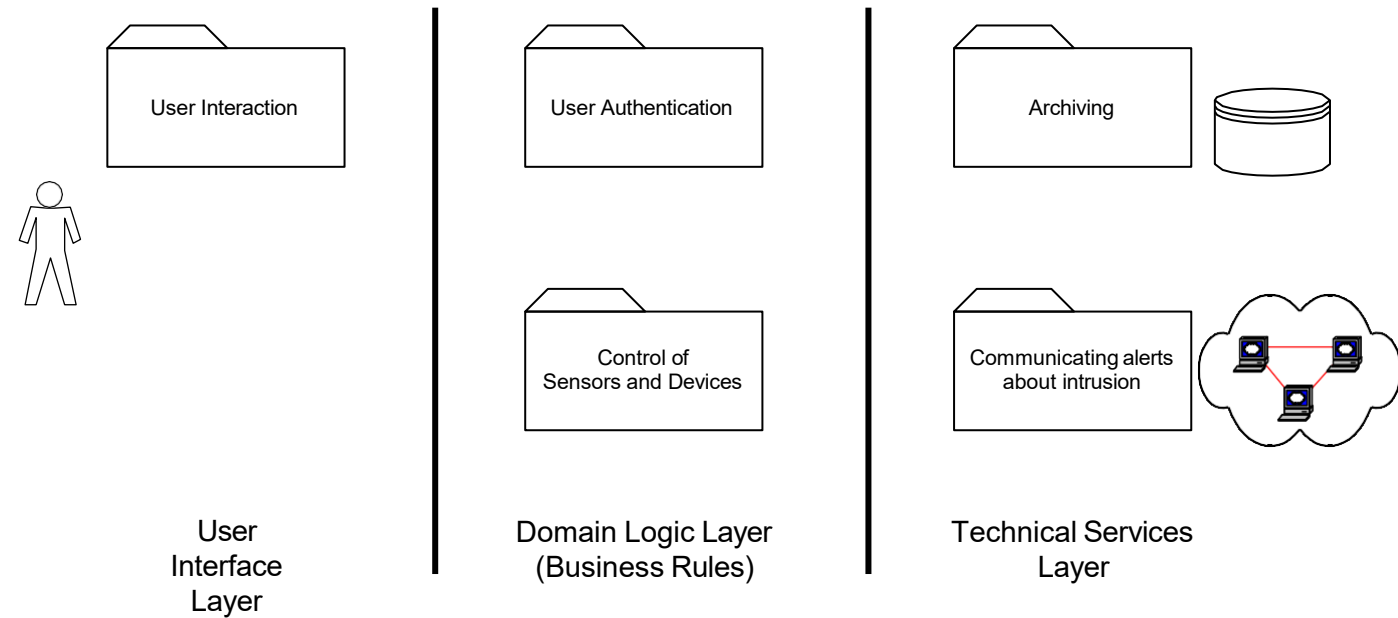
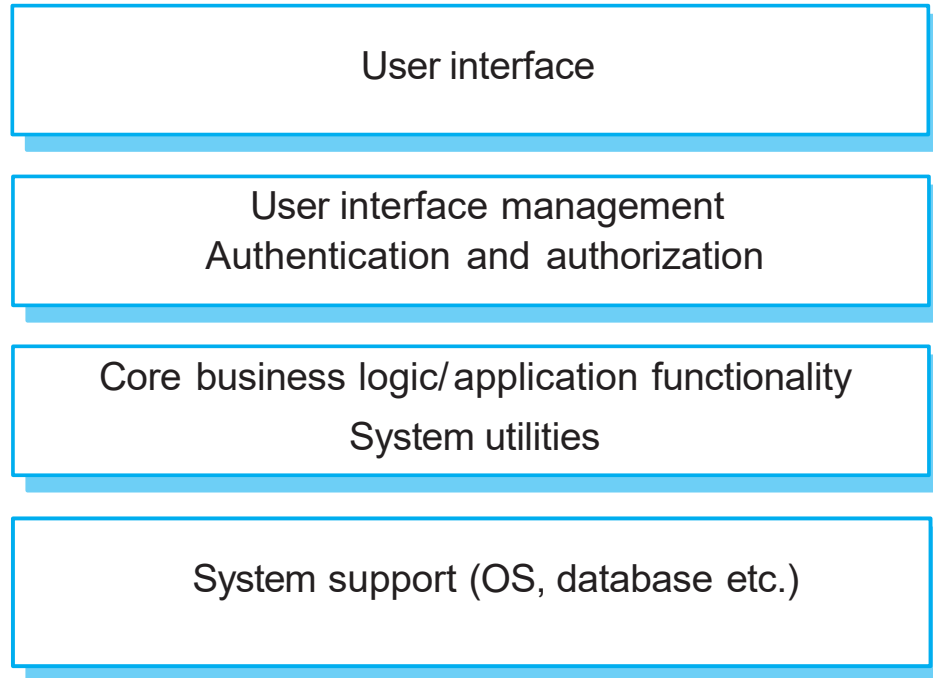
⚙ MVC is a very useful architectural style for the web



Layered architecture

- Breaks a system into **layers**, each with a clear role.
- Each layer provides **services to the layer above**.
- Makes systems **easier to develop and maintain**.
- A change in one layer mostly affects only its **neighbouring layers**.
- But sometimes it feels **unnatural** to force a system into layers

Layered architecture



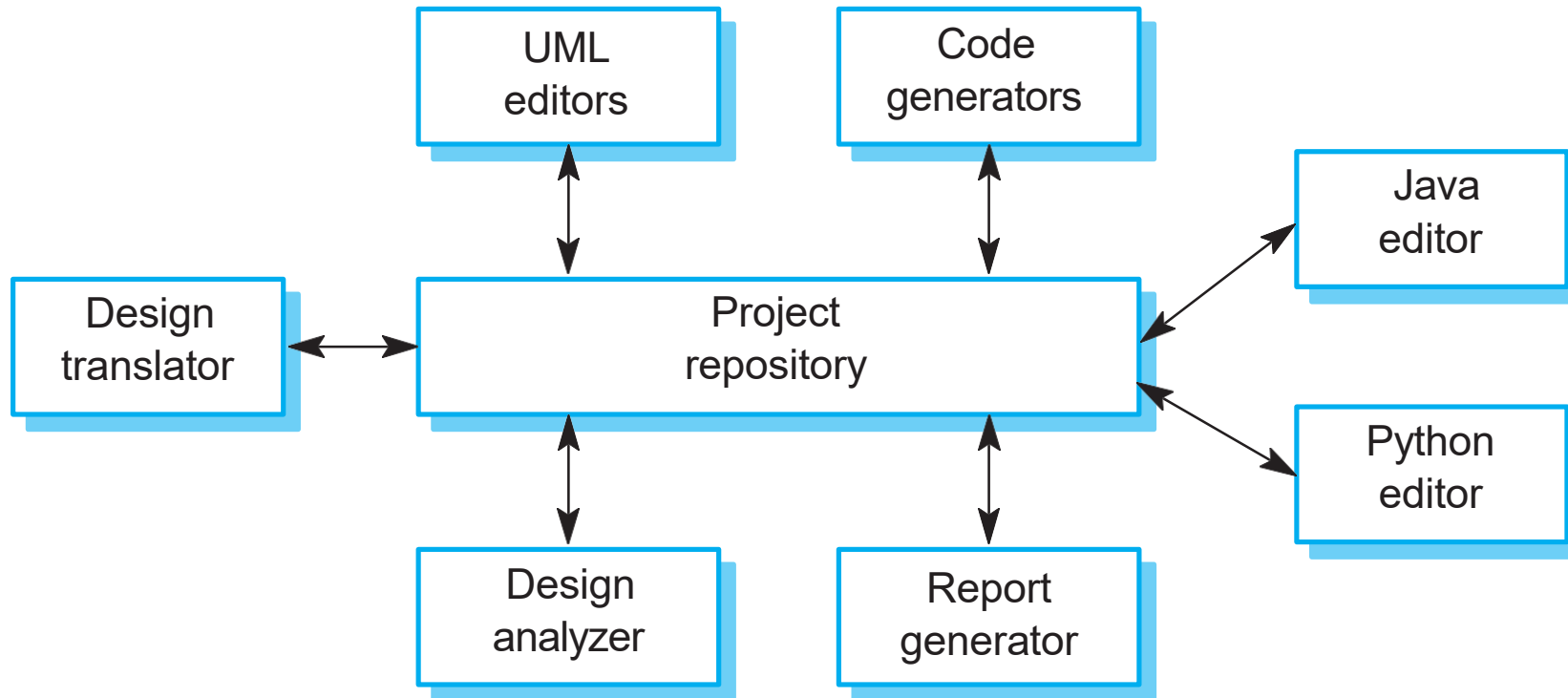
The Layered architecture pattern

Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system.
Example	A layered model of a system for sharing copyright documents held in different libraries.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

Repository architecture

- Provides a **central data store** accessed by multiple subsystems.
- Subsystems exchange data either via:
 - **Central repository** (shared database)
 - **Distributed repositories** (each subsystem owns its data, shares when needed)
- Common when **large volumes of data** must be managed and shared.
- Supports **integration and consistency**, but can become a **single point of failure**.

A repository architecture for an IDE



The Repository pattern

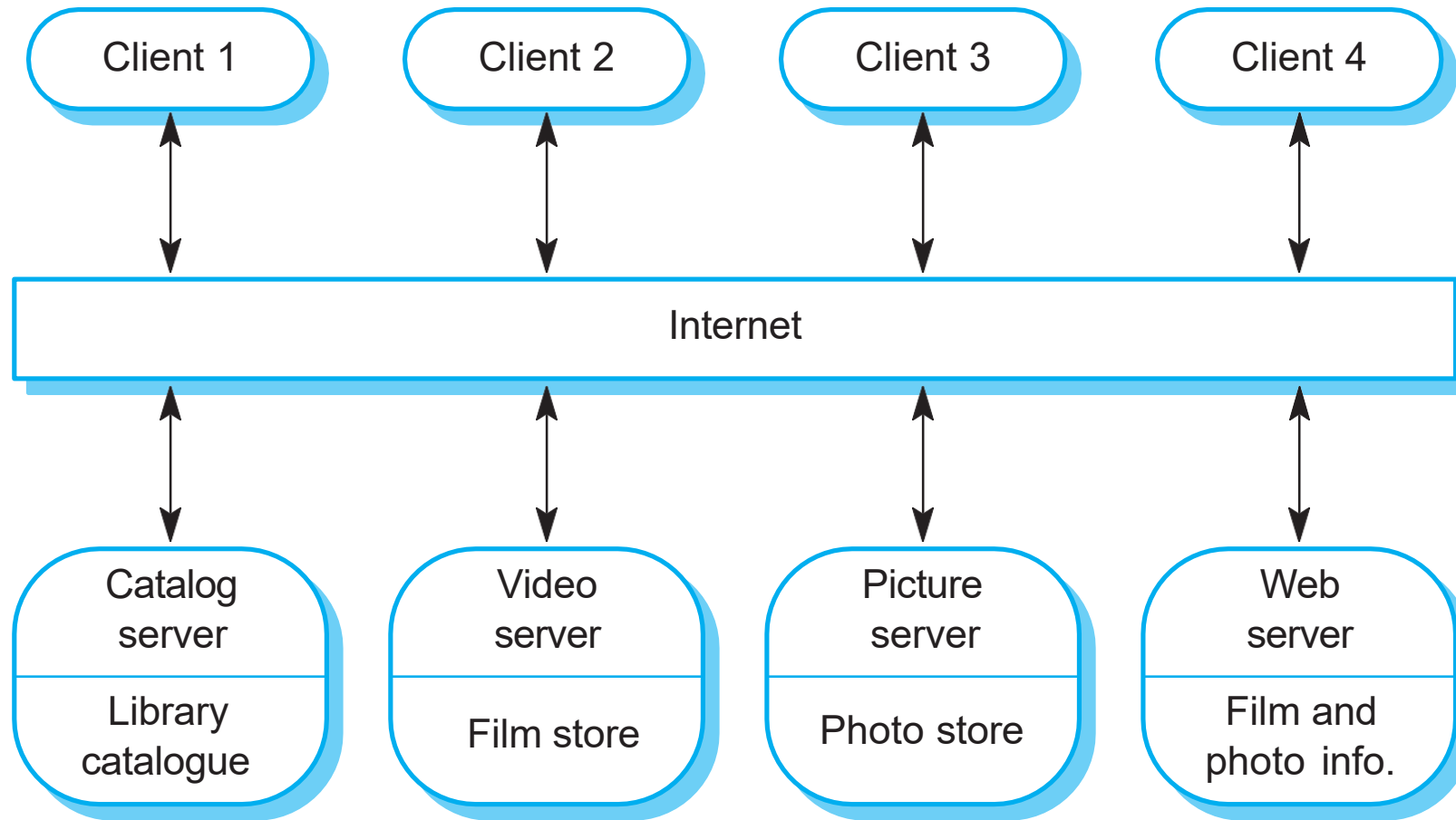
Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	An IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

Client-server architecture

- A **distributed model** where tasks are split between clients and servers.
- **Servers** → provide services (e.g., data storage, printing, authentication).
- **Clients** → request and use these services.
- **Network** → connects clients to servers.

Can also run on a **single machine** (client and server together)

A client–server architecture for a film library



The Client–server pattern

Name	Client-server
Description	In a client–server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	A film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

Application architectures

- ⚙ Application systems are designed to meet an organisational need.
- ⚙ As businesses have much in common, their application systems also tend to have a common architecture that reflects the application requirements.
- ⚙ A generic application architecture is an architecture for a type of software system that may be configured and adapted to create a system that meets specific requirements.

Examples of application types

⚙ Data processing applications

- ☞ Data driven applications that process data in batches without explicit user intervention during the processing.

⚙ Transaction processing applications

- ☞ Data-centred applications that process user requests and update information in a system database.

⚙ Event processing applications

- ☞ Applications where system actions depend on interpreting events from the system's environment.

⚙ Language processing applications

- ☞ Applications where the users' intentions are specified in a formal language that is processed and interpreted by the system.

Transaction processing systems

- ⚙️ Process user requests for information from a database or requests to update the database.
- ⚙️ From a user perspective a transaction is:
 - 👉 Any coherent sequence of operations that satisfies a goal;
 - 👉 For example - find the times of flights from London to Paris.
- ⚙️ Users make asynchronous requests for service which are then processed by a transaction manager.

Transaction processing application structure

